

EJERCICIOS

Ejercicio 1

Implementa el siguiente algoritmo.

```
1  import cv2
2  import numpy as np
3
4  # Captura de video (0 = cámara web)
5  cap = cv2.VideoCapture(0)
6
7  # Función auxiliar para poner título con sombra
8  def agregar_titulo(img, texto):
9      img_copy = img.copy()
10     # Texto negro (sombra)
11     cv2.putText(img_copy, texto, (11, 26), cv2.FONT_HERSHEY_SIMPLEX,
12                0.8, (0, 0, 0), 3, cv2.LINE_AA)
13     # Texto blanco encima
14     cv2.putText(img_copy, texto, (10, 25), cv2.FONT_HERSHEY_SIMPLEX,
15                0.8, (255, 255, 255), 2, cv2.LINE_AA)
16     return img_copy
17
18  while True:
19      ret, frame = cap.read()
20      if not ret:
21          break
22
23      # Redimensionar para que todas las imágenes tengan el mismo tamaño
24      frame = cv2.resize(frame, (320, 240))
25
26      # Escala de grises
27      gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
28
29      # Operaciones con matrices
30      brillo = cv2.add(gray, 50)
31      oscuro = cv2.subtract(gray, 50)
32      negativo = 255 - gray
33      rotado = cv2.transpose(gray)
34
35      # Convertir a BGR para unir
36      gray_bgr = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
37      brillo_bgr = cv2.cvtColor(brillo, cv2.COLOR_GRAY2BGR)
38      oscuro_bgr = cv2.cvtColor(oscuro, cv2.COLOR_GRAY2BGR)
39      negativo_bgr = cv2.cvtColor(negativo, cv2.COLOR_GRAY2BGR)
40      rotado_bgr = cv2.cvtColor(rotado, cv2.COLOR_GRAY2BGR)
41
42      # Redimensionar el rotado
43      rotado_bgr = cv2.resize(rotado_bgr, (320, 240))
44
```

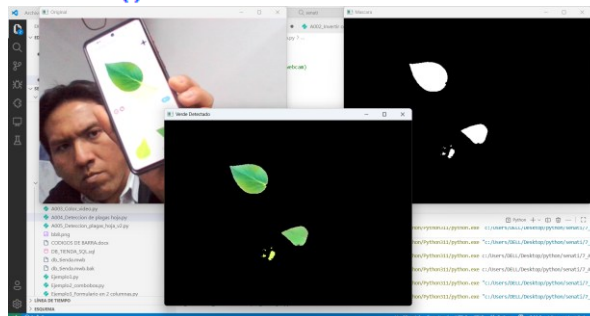
```
45 # Agregar títulos
46 frame = agregar_titulo(frame, "Original (Color)")
47 gray_bgr = agregar_titulo(gray_bgr, "Escala de grises")
48 brillo_bgr = agregar_titulo(brillo_bgr, "Brillo +50")
49 oscuro_bgr = agregar_titulo(oscuro_bgr, "Oscuro -50")
50 negativo_bgr = agregar_titulo(negativo_bgr, "Negativo")
51 rotado_bgr = agregar_titulo(rotado_bgr, "Rotado (Traspuesta)")
52
53 # Crear mosaico
54 fila1 = np.hstack((frame, gray_bgr, brillo_bgr))
55 fila2 = np.hstack((oscuro_bgr, negativo_bgr, rotado_bgr))
56 mosaico = np.vstack((fila1, fila2))
57
58 # Mostrar mosaico
59 cv2.imshow("Transformaciones con Algebra Lineal", mosaico)
60
61 # Salir con ESC
62 if cv2.waitKey(30) == 27:
63     break
64
65 cap.release()
66 cv2.destroyAllWindows()
```



Ejercicio 2

Implementa el siguiente algoritmo.

```
1  import cv2
2  import numpy as np
3
4  # Captura desde cámara (0 = webcam)
5  cap = cv2.VideoCapture(0)
6
7  while True:
8      ret, frame = cap.read()
9      if not ret:
10         break
11
12     # --- Convertir de BGR a HSV ---
13     hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
14
15     # --- Rango de color verde en HSV ---
16     # (estos valores pueden ajustarse según la iluminación real)
17     verde_bajo = np.array([35, 40, 40]) # tono, saturación, valor
18     verde_alto = np.array([85, 255, 255])
19
20     # --- Crear máscara para detectar verdes ---
21     mask = cv2.inRange(hsv, verde_bajo, verde_alto)
22
23     # --- Aplicar máscara a la imagen original ---
24     resultado = cv2.bitwise_and(frame, frame, mask=mask)
25
26     # --- Mostrar ventanas ---
27     cv2.imshow("Original", frame)
28     cv2.imshow("Verde Detectado", resultado)
29     cv2.imshow("Mascara", mask)
30
31     # Salir con ESC
32     if cv2.waitKey(30) == 27:
33         break
34
35 cap.release()
36 cv2.destroyAllWindows()
```



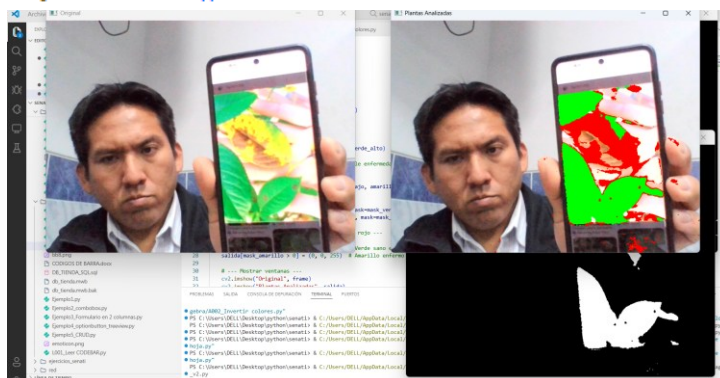
Ejercicio 3

Implementa el siguiente algoritmo.

```

1  import cv2
2  import numpy as np
3
4  cap = cv2.VideoCapture(0)
5  while True:
6      ret, frame = cap.read()
7      if not ret:
8          break
9      hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
10
11     # --- Rango de color verde (sano) ---
12     verde_bajo = np.array([35, 40, 40])
13     verde_alto = np.array([85, 255, 255])
14     mask_verde = cv2.inRange(hsv, verde_bajo, verde_alto)
15
16     # --- Rango de color amarillo/marrón (posible enfermedad) ---
17     amarillo_bajo = np.array([15, 40, 40])
18     amarillo_alto = np.array([35, 255, 255])
19     mask_amarillo = cv2.inRange(hsv, amarillo_bajo, amarillo_alto)
20
21     # --- Crear imágenes resaltadas ---
22     res_verde = cv2.bitwise_and(frame, frame, mask=mask_verde)
23     res_amarillo = cv2.bitwise_and(frame, frame, mask=mask_amarillo)
24
25     # --- Combinar: verde en verde, amarillo en rojo ---
26     salida = frame.copy()
27     salida[mask_verde > 0] = (0, 255, 0) # Verde sano en verde
28     salida[mask_amarillo > 0] = (0, 0, 255) # Amarillo enfermo en rojo
29
30     # --- Mostrar ventanas ---
31     cv2.imshow("Original", frame)
32     cv2.imshow("Plantas Analizadas", salida)
33     cv2.imshow("Mascara Verde", mask_verde)
34     cv2.imshow("Mascara Amarillo", mask_amarillo)
35
36     # Salir con ESC
37     if cv2.waitKey(30) == 27:
38         break
39 cap.release()
40 cv2.destroyAllWindows()

```



EJERCICIO INTEGRACIÓN

A partir de lo ejemplos desarrollados anteriormente plantea o escoge una de las siguientes propuestas y desarrollar una solución creativa que combine la visión por computadora con un problema real del entorno. (Puedes utilizar cualquier librería que permita dar solución al problema propuesto)

Propuestas de desarrollo:

1. **Detección temprana de enfermedades en hojas de cultivo:** identificar hojas sanas y enfermas mediante rangos de color.
2. **Clasificación de frutas maduras e inmaduras:** determinar el estado de maduración según su color.
3. **Sistema de alerta para seguridad vial:** reconocer en tiempo real los colores del semáforo.
4. **Detección de lunares malignos o anomalías en piel:** analizar variaciones de color en la piel como apoyo a la telemedicina.
5. **Reconocimiento de residuos para reciclaje:** clasificar materiales (vidrio, plástico, orgánico, etc.) a partir de sus colores característicos.